

Revisiting the Compact Tail Assignment Model: Corrections and an Exact Event-Based Maintenance Reformulation

Viktor Borodin^a, Arun Kumar Pappala^b

^a*Taras Shevchenko National University, Kyiv, Ukraine*

^b*Singapore University of Technology and Design, Singapore*

Abstract

This paper revisits the compact mixed-integer linear programming model for the Tail Assignment Problem (TAP) introduced by Khaled et al. (2018). We make three contributions. First, we show that the basic continuity/turn-time criteria (C2–C4; model constraints (3)–(6) in Khaled et al.) admit a simultaneous-departure corner case that is mathematically feasible but physically infeasible, and we close this gap with explicit non-overlap constraints. Second, we identify and formally correct a flaw in the paper’s cumulative-flight-time constraint C 11 (constraint (13) in Khaled et al.): the original formulation allows maintenance intervals to be incorrectly relaxed. We show that a two-row endpoint split is also insufficient and give an exact cumulative-state formulation. We give a numeric counterexample, evaluated against the real built constraint objects at two adversarial check patterns, in which the original and endpoint-split formulations each fail on a different pattern while the ordered formulation rejects both, and we independently verify every reported solution by replaying its maintenance counters outside the model. Third, we propose an alternative to Khaled et al.’s set of constraints for the

same TAP problem with type-A maintenance: an ordered, day-free event-based formulation that uses aircraft-local prefix state without a day-pair window. We prove the reformulation is correct, resolve the counterexamples mentioned, evaluate it and show that it uses substantially fewer variables than the day-indexed model and have a speed-up on most cases. All code, instances, and result tables accompany the project.

Keywords: Tail assignment, Aircraft routing, Maintenance scheduling, Integer programming, Mixed-integer programming corrections

1. Introduction

The airline planning process is traditionally decomposed into a sequence of sub-problems: timetable design, fleet assignment, crew rostering, tail assignment, and recovery planning (Barnhart et al., 2003; Gopalan and Talluri, 1998). The *tail assignment problem* (TAP)—assigning a specific aircraft (identified by its tail number) to each flight leg—is the penultimate planning step and is tightly coupled with aircraft maintenance scheduling. Poor tail assignment decisions directly inflate operational costs through unnecessary ferry (repositioning) flights, suboptimal maintenance routing, and safety-margin violations.

In addition to the basic assignment and continuity requirements, the TAP must respect aircraft maintenance constraints. The most common maintenance type is *type-A*, which requires a maintenance check after a maximum flying-time interval. Type-A maintenance is typically performed at a limited set of airports, and the TAP must ensure that each aircraft is routed to a maintenance-capable airport before its flying-time limit is exceeded. The

type-A maintenance requirement is a key operational constraint in the TAP and has been studied extensively in the literature.

Khaled et al. (2018) proposed a compact 0–1 linear programming formulation for the TAP without maintenance, featuring only $O(n_f n_p)$ binary variables (where n_f is the number of flights and n_p the fleet size). This is a significant reduction over the classic multi-commodity flow model of Levin (1971), which requires $O(n_f^2 n_p)$ variables. Further, Khaled et al. (2018) extended the model to include type-A maintenance requirements, which are defined by a maximum flying-time interval between maintenance checks, that adds other $O(n_f n_p D)$ binary variables, where D is the number of calendar days in the planning horizon. Computational experiments reported in that paper show that instances with up to 1 500 flight legs and 40 aircraft can be solved to certified optimality within one hour on a standard workstation.

Our contributions

The present paper makes three contributions that check and correct those results:

1. **Basic feasibility correction (C2–C4 / constraints (3)–(6)).** We show that the strict-time balance definition used in the compact basic model admits a simultaneous-departure corner case: one aircraft can be assigned to two flights departing at the same airport and same time without violating the balance inequalities. We formalize this counter-example and enforce physical executability by adding explicit pairwise non-overlap constraints.

Our non-overlap correction retains the $O(n_f n_p)$ binary-variable space but can add $O(n_f^2 n_p)$ conflict inequalities in the worst case. In prac-

tical timetables, only temporally conflicting pairs are instantiated; the resulting constraint count is therefore governed by the sparse conflict set $\mathcal{O}$ defined in Section 3.

2. **Constraint (11) correction.** We show that the cumulative-flight-time maintenance constraint (constraint (13) in Khaled et al. (2018)) contains a logical flaw in opening and one-endpoint intervals, and in the handling of consecutive flights/maintenances. We also show by counterexample that a natural two-row endpoint split does not fully repair the model. We therefore formulate an exact cumulative-state recursion that propagates and resets the flight-hour counter without day-pair ambiguity.
3. **Event-based reformulation.** The day-indexed type-A model above still repeats a calendar-day index across every maintenance variable. We give a complete, day-free reformulation of the same type-A C1–C14 requirements, in which each flight’s own timestamp supplies all calendar information and maintenance is represented as a virtual flight triggered by, and indexed on, the preceding real flight rather than by calendar day. This removes the day dimension from the maintenance model’s size and lets the flight-hour counter propagate exactly along each aircraft’s actual route, with no day-pair window and therefore no possibility of the split-form failure mode proved for C11.

Paper organisation

Section 2 reviews related work. Section 3 restates the basic TAP model and proves a C2–C4 corner-case correction. Section 4 presents the maintenance-extended model, the Constraint (11) correction, and the induced interpreta-

tion updates for C5–C11 under the corrected basic model. Section 5 gives the day-free event-based reformulation of the type-A requirements and proves its correctness directly. Section 6 reports the numeric C11 counterexample and the benchmarking results of the legacy and ordered models, and Section 7 concludes.

2. Literature Review

2.1. Mathematical programming formulations

Three main modelling paradigms underlie TAP research:

Set partitioning / column generation.. The non-compact formulation enumerates *strings* (feasible flight sequences per aircraft) as 0–1 variables. Because the number of strings is exponential, column generation is required (Lavoie et al., 1988; Minoux, 1984; Barnhart et al., 1998; Sarac et al., 2006). Combining column generation with branch-and-bound (*branch-and-price*) gives exact solutions but at significant algorithmic overhead. Robust extensions appear in Borndörfer et al. (2010) and Froyland et al. (2013).

Multi-commodity flow.. Levin (1971) introduced the compact multi-commodity flow formulation with $O(n_f^2 n_p)$ binary variables. This remains the standard benchmark for compact models and is the primary comparison point in Khaled et al. (2018).

Time-space networks.. Hane et al. (1995) proposed the time-space network for the *fleet assignment problem*. Nodes represent airport–time events; arcs encode ground waits, flights, and overnight stays. Extensions to aircraft routing appear in Clarke et al. (1996). Preprocessing by node aggregation

and island isolation reduces model size (Hane et al., 1995; Sherali et al., 2006). Daily and weekly maintenance routing variants are studied in Liang et al. (2011) and Liang and Chaovalitwongse (2012); the latter also integrates fleet assignment decisions, but only approximate solutions are obtained via variable fixing.

Nonlinear / lifted formulations.. Haouari et al. (2012) present a compact formulation with nonlinear constraints that are linearised via the Reformulation–Linearisation Technique (RLT). The largest instances solved involve 344 flights, an order of magnitude below the instances handled by the model of Khaled et al. (2018).

2.2. MILP alternatives to the Khaled compact model

Beyond the compact binary assignment model of Khaled et al. (2018), several mixed-integer alternatives are relevant for exact TAP modelling and large-scale implementation:

- **Arc-flow MILP (time-indexed or event-indexed):** decision variables are attached to feasible flight-connection arcs instead of flight–aircraft assignment pairs, often yielding stronger network-structure constraints (Levin, 1971; Hane et al., 1995; Sherali et al., 2006).
- **Route-based master MILP (set partitioning):** aircraft duties are represented as columns in a master MILP, typically solved with branch-and-price; this is exact but algorithmically heavier (Barnhart et al., 1998; Sarac et al., 2006; Klabjan, 2005).

- **Decomposed MILP frameworks:** Benders- or Dantzig–Wolfe-style decompositions separate assignment/routing from maintenance-resource feasibility, improving scalability on larger fleets (Cordeau et al., 2001; Mercier et al., 2005; Desaulniers et al., 1997).
- **Two-stage or rolling-horizon MILP:** first assign flights, then repair/optimize maintenance and turnaround feasibility in a second MILP, trading global optimality for tractable runtime (Clarke et al., 1996; Liang et al., 2011; Liang and Chaovalitwongse, 2012).
- **Robust/scenario MILP variants:** delay and disruption uncertainty is modelled through multiple scenarios with non-anticipativity-style linking constraints, producing schedules that are less brittle operationally (Borndörfer et al., 2010; Froyland et al., 2013).

In this taxonomy, the Khaled model is best viewed as a compact, directly solvable baseline that is simple to implement and benchmark, while the alternatives above trade model size, decomposition complexity, and robustness for stronger scalability or richer operational realism.

2.3. Comparison scope and fairness

For an empirical comparison to be methodologically fair, competing models should be evaluated under a common instance family, objective definition, maintenance semantics, and solver-budget protocol. In TAP, this condition is often not met across compact MILP, branch-and-price, Benders-decomposed, and robust/scenario formulations because these approaches typically optimize different decision scopes or uncertainty assumptions. Accordingly, this

paper uses Khaled et al. (2018) as the direct compact baseline for instance-matched comparisons, and treats other MILP families as complementary reference paradigms rather than strict one-to-one benchmarks.

2.4. Maintenance constraints

Maintenance scheduling within aircraft routing has been studied since the early work of Clarke et al. (1997). The column-generation framework of Barnhart et al. (1998) can embed type-A constraints through constrained shortest-path pricing. Cordeau et al. (2001) apply Benders decomposition to simultaneous routing and crew scheduling with non-linear maintenance resources. Lacasse-Guay et al. (2010) consider maintenance under different business-process models. We focus on the type-A flight-hour case and its exact event-based reformulation.

2.5. Heuristic approaches

Constructive heuristics for TAP are comparatively under-studied relative to exact methods. Grönkvist (2005) provides a comprehensive treatment including local-search improvements. Heuristic and metaheuristic solution methods are outside the scope of the present exact-formulation study and are not evaluated here.

2.6. Position of the present paper

The compact formulation of Khaled et al. (2018) represents the reference compact baseline for exact, cyclic-constraint-free TAP in this study. The present work builds directly upon it, providing: (i) a diagnosis of presented in Khaled et al. (2018) C1-C11 and an exact state-based replacement, (ii)

Table 1: Notation summary.

Symbol	Meaning
$\mathcal{F} = \{1, \dots, n_f\}$	Set of flight legs
$\mathcal{P} = \{1, \dots, n_p\}$	Set of aircraft (tail numbers)
$\mathcal{A} = \{1, \dots, n_k\}$	Set of airports
$a_i^\uparrow, a_i^\downarrow$	Departure / arrival airport of flight i
$t_i^\uparrow, t_i^\downarrow$	Departure / arrival time of flight i (minutes)
$d_i, \bar{d}_i$	Departure / arrival day derived from flight times
Δt	Minimum ground turn time (minutes)
c_{ij}	Assignment cost of flight i to aircraft j
a_j^0	Initial position airport of aircraft j
$F_k^\downarrow(\theta)$	$\{i \in \mathcal{F} : a_i^\downarrow = k, t_i^\downarrow \leq \theta - \Delta t\}$
$F_k^\uparrow(\theta)$	$\{i \in \mathcal{F} : a_i^\uparrow = k, t_i^\uparrow < \theta\}$

a new reformulation of the event-based model that resolves provided counterexamples, and gives improved computational performance, and (iii) an instance-matched benchmark of the implemented exact formulations.

3. The Basic Tail Assignment Model

We restate the compact model of Khaled et al. (2018) using the notation of Table 1. This section covers the model without maintenance constraints; they are introduced in Section 4.

3.1. Decision variables and objective

Let

$$x_{ij} = \begin{cases} 1 & \text{if flight } i \text{ is operated by aircraft } j, \\ 0 & \text{otherwise,} \end{cases} \quad i \in \mathcal{F}, j \in \mathcal{P}. \quad (1)$$

The objective minimises total assignment cost:

$$\min \sum_{i \in \mathcal{F}} \sum_{j \in \mathcal{P}} c_{ij} x_{ij}. \quad (2)$$

3.2. Constraints

C1 — Flight coverage.. Each flight must be assigned to exactly one aircraft:

$$\sum_{j \in \mathcal{P}} x_{ij} = 1, \quad \forall i \in \mathcal{F}. \quad (3)$$

C2, C3 — Equipment continuity (home/non-home airports).. For any aircraft j , any non-home airport $k \neq a_j^0$, and any flight i departing from k , aircraft j can be assigned to flight i only if it has already landed at k early enough, but at the home airport ($k = a_j^0$), one free unit is available at time zero

$$\sum_{i' \in F_k^\downarrow(t_i^\uparrow)} x_{i'j} - \sum_{i' \in F_k^\uparrow(t_i^\uparrow)} x_{i'j} \geq x_{ij}, \quad \forall j \in \mathcal{P}, k \in \mathcal{A} \setminus \{a_j^0\}, i \in \mathcal{F} : a_i^\uparrow = k. \quad (4)$$

$$\sum_{i' \in F_k^\downarrow(t_i^\uparrow)} x_{i'j} - \sum_{i' \in F_k^\uparrow(t_i^\uparrow)} x_{i'j} \geq x_{ij} - 1, \quad \forall j \in \mathcal{P}, k = a_j^0, i \in \mathcal{F} : a_i^\uparrow = k. \quad (5)$$

Integrality..

$$x_{ij} \in \{0, 1\}, \quad \forall i \in \mathcal{F}, j \in \mathcal{P}. \quad (6)$$

3.3. Validity

Constraints (4)–(5) are intended to encode both equipment continuity (C2 of Khaled et al. (2018)) and turn-time feasibility (C4). The set $F_k^\downarrow(t_i^\uparrow)$ counts only arrivals at least Δt minutes before the departure of flight i , so a prior arrival can supply flight i only after the required turn time. This mechanism is valid when departures are strictly ordered, but it does not prevent two equal-time departures from using the same available aircraft, as shown below.

Proposition 1 (Claimed in Khaled et al. 2018, Proposition 1). *Any feasible solution to (3)–(6) corresponds to a feasible flight plan satisfying conditions C1–C4.*

The proposition is not valid as stated when distinct flights may have equal departure times. The following counterexample identifies the missing case.

3.4. Counter-example with simultaneous departures

The balance constraints (4)–(5) use $F_k^\uparrow(\theta) = \{i \in \mathcal{F} : a_i^\uparrow = k, t_i^\uparrow < \theta\}$ with a strict inequality in departure time. If two flights leave the same airport at the same time, neither flight appears in the other’s departure-balance term. This can make an assignment feasible in (3)–(6) while being physically impossible as a flight plan.

Example 1 (Feasible to (3)–(6), infeasible in practice). Consider one aircraft j with initial airport $a_j^0 = B$ and turn time $\Delta t = 30$ minutes. Let the three

flights be:

$$\begin{aligned} i_0 : B \rightarrow A, \quad t_{i_0}^\uparrow = 500, \quad t_{i_0}^\downarrow = 550, \\ i_1 : A \rightarrow B, \quad t_{i_1}^\uparrow = 600, \quad t_{i_1}^\downarrow = 700, \\ i_2 : A \rightarrow C, \quad t_{i_2}^\uparrow = 600, \quad t_{i_2}^\downarrow = 700. \end{aligned}$$

Set $x_{i_0j} = x_{i_1j} = x_{i_2j} = 1$.

Coverage (3) is satisfied (single aircraft, all flights assigned). Flight i_0 satisfies the home-airport constraint (5), because no flight has arrived at or departed from B before time 500. For i_1 at the non-home airport A:

$$F_A^\downarrow(t_{i_1}^\uparrow) = \{i_0\}, \quad F_A^\uparrow(t_{i_1}^\uparrow) = \emptyset,$$

because i_0 departs from B and $t_{i_2}^\uparrow = 600 \not\leq 600$. Hence constraint (4) gives

$$\sum_{i' \in F_A^\downarrow(t_{i_1}^\uparrow)} x_{i'j} - \sum_{i' \in F_A^\uparrow(t_{i_1}^\uparrow)} x_{i'j} = 1 - 0 = 1 \geq x_{i_1j} = 1.$$

Symmetrically for i_2 , $t_{i_1}^\uparrow = 600 \not\leq 600$, so the same calculation gives $1 \geq x_{i_2j} = 1$. Therefore (3)–(6) are feasible.

However, the resulting plan is physically infeasible: one aircraft cannot operate both i_1 and i_2 simultaneously at time 600.

Implication.. In implementations, this corner case is eliminated by adding pairwise non-overlap constraints for time-overlapping flights on the same aircraft, $x_{i_1j} + x_{i_2j} \leq 1$.

Corrected C1–C3 feasibility criterion.. Define $\mathcal{O}$ as the set of flight pairs that cannot be operated by one aircraft because their operating intervals, including the turn-time buffer, overlap:

$$\mathcal{O} = \left\{ \{i_1, i_2\} \subseteq \mathcal{F} : [t_{i_1}^\uparrow, t_{i_1}^\downarrow + \Delta t) \cap [t_{i_2}^\uparrow, t_{i_2}^\downarrow + \Delta t) \neq \emptyset \right\}. \quad (7)$$

For every pair in this conflict set, impose

$C4$ — *Overlap constraint*..

$$x_{i_1j} + x_{i_2j} \leq 1, \quad \forall \{i_1, i_2\} \in \mathcal{O}, \forall j \in \mathcal{P}. \quad (8)$$

The corrected core model is therefore (3)–(5), (8), and (6). This preserves the original $n_f n_p$ binary-variable space while restoring physical executability. It does not, in general, preserve the linear constraint count: the overlap family contains $|\mathcal{O}|n_p$ inequalities.

3.5. Model size

Proposition 2 (Khaled et al. 2018, Proposition 2). *The original model (2)–(6) contains $n_f n_p$ binary decision variables and $O(n_f n_p)$ constraints.*

The corrected model retains $n_f n_p$ binary variables and has $O(n_f n_p + |\mathcal{O}|n_p)$ constraints. In the worst case $|\mathcal{O}| = O(n_f^2)$, although timetable sparsity normally makes the conflict set much smaller. The original variable count compares favourably with Levin’s $O(n_f^2 n_p)$ -variable model; Khaled et al. (2018) also report that the time-space network formulation uses approximately 1.25–2 times as many variables and constraints on their test instances.

4. The Maintenance-Extended Model and Correction of Constraint (11)

4.1. Maintenance problem definition

Following Khaled et al. (2018), we consider type-A checks triggered after at most $T_{\max}^A$ cumulative flight hours. Checks take place at night after the last flight on a given day in an airport with maintenance capacity.

Additional notation is collected in Table 2.

Table 2: Additional notation for the maintenance model.

Symbol	Meaning
$\mathcal{D} = \{1, \dots, n_d\}$	Planning days
d_i	Departure day of flight i
$\bar{d}_i$	Arrival day of flight i
$\mathcal{M} \subseteq \mathcal{A}$	Maintenance-capable airports
$\text{mcap}_{m,d}$	Maintenance capacity of airport m on day d
mc_{ijd}	Cost of night maintenance at arrival airport of flight i , aircraft j , day d
$F_d^\downarrow(m)$	Flights arriving at airport m on day d
$F(d, i)$	$\{i' : d_{i'} = d, t_{i'}^\uparrow > t_i^\downarrow\}$
$T_{\max}^A$	Max cumulative flight minutes before type-A check
$d_{\max}$	Max calendar-day interval between checks
$M_{dd'}$	Big- M for day pair (d, d')
$F_{d,d'}$	Flights with departure day in $(d, d']$

4.2. Additional decision variables

$$z_{ijd} = \begin{cases} 1 & \text{night maintenance at arrival airport of } i, \text{ aircraft } j, \text{ day } d, \\ 0 & \text{else;} \end{cases}$$

$$y_{jd} = \begin{cases} 1 & \text{maintenance of aircraft } j \text{ on day } d, \\ 0 & \text{else,} \end{cases}$$

The variable z_{ijd} is instantiated only when flight i arrives at a maintenance-capable airport on day d . Thus its admissible domain is $i \in \bigcup_{m \in \mathcal{M}} F_d^\downarrow(m)$. We assume in this type-A model that a night check fits within the overnight

ground period.

4.3. Extended model

The objective becomes:

$$\min \sum_{i \in \mathcal{F}} \sum_{j \in \mathcal{P}} c_{ij} x_{ij} + \sum_{d \in \mathcal{D}} \sum_{m \in \mathcal{M}} \sum_{i \in F_d^\downarrow(m)} \sum_{j \in \mathcal{P}} m c_{ijd} z_{ijd}. \quad (9)$$

Constraints (3)–(5) are retained. Additional constraints:

C5 — Maintenance blocks same-day subsequent flights..

$$z_{ijd} + x_{i'j} \leq 1, \quad \forall d \in \mathcal{D}, m \in \mathcal{M}, i \in F_d^\downarrow(m), i' \in F(d, i), j \in \mathcal{P}. \quad (10)$$

C6 — Maintenance requires assignment..

$$x_{ij} \geq z_{ijd}, \quad \forall d \in \mathcal{D}, m \in \mathcal{M}, i \in F_d^\downarrow(m), j \in \mathcal{P}. \quad (11)$$

C7 — Capacity..

$$\sum_{i \in F_d^\downarrow(m)} \sum_{j \in \mathcal{P}} z_{ijd} \leq \text{mcap}_{m,d}, \quad \forall d \in \mathcal{D}, m \in \mathcal{M}. \quad (12)$$

C8 — Aggregation ($z \rightarrow y$)..

$$\sum_{m \in \mathcal{M}} \sum_{i \in F_d^\downarrow(m)} z_{ijd} = y_{jd}, \quad \forall d \in \mathcal{D}, j \in \mathcal{P}. \quad (13)$$

C9 — Maximum-spacing constraint..

$$\sum_{r \in [d, d']} y_{jr} \geq 1, \quad \forall d, d' \in \mathcal{D} \text{ s.t. } d' - d = d_{\max} - 1, \forall j \in \mathcal{P}. \quad (14)$$

For the type-A-only model studied in this paper, C9 is not an additional calendar-spacing requirement: type-A maintenance is governed by the exact flight-hour state in C13. The displayed row is retained as the generic day-indexed maintenance notation and becomes active only for a calendar-time maintenance type, which is outside the scope of this paper.

C10 — At most one maintenance event per aircraft per day..

$$\sum_{m \in \mathcal{M}} \sum_{i \in F_d^\downarrow(m)} z_{ijd} \leq 1, \quad \forall j \in \mathcal{P}, d \in \mathcal{D}. \quad (15)$$

For binary y , C10 is implied by C8; it is retained because it makes the one-event-per-day rule explicit and remains useful in implementations that relax or disaggregate the aggregation equality.

The original paper’s cumulative-flight-time constraint is:

$$\sum_{i \in F_{d,d'}} x_{ij} \tilde{t}_i \leq T_{\max}^A + M_{dd'} (2 - y_{jd} - y_{jd'}) + M_{dd'} \sum_{r \in [d+1, d'-1]} y_{jr}, \quad (16)$$

for all $j \in \mathcal{P}$, all pairs (d, d') with $2 \leq d' - d \leq d_{\max} - 1$, $d < n_d$, where $\tilde{t}_i$ is the flight duration of leg i .

4.4. Impact of corrected C2–C4 feasibility on C5–C10

The correction introduced in Section 3 (non-overlap constraint (8)) changes the admissible assignment set for x , and therefore the interpretation of all maintenance constraints layered on top of x .

C5.. No algebraic change is required in (10); however, with (8) active, the blocking relation now operates on physically realizable trajectories only. This removes spurious cases where maintenance placement was evaluated on infeasible simultaneous departures.

C6–C10.. Constraints (11)–(15) remain structurally unchanged. C7 is inactive in the type-A-only model. Their feasible region is tightened indirectly because z and y are linked to a strictly smaller, physically valid x -space.

C9 and onward.. Hour-accumulation constraints are evaluated over a physically executable x -space once (8) is imposed. This correction is necessary, but it does not by itself establish that a particular C13 algebraic form correctly propagates the maintenance counter; that property is analysed below.

In summary, the C2–C4 correction primarily changes the domain over which the maintenance constraints operate. It is independent of the separate algebraic correction required for cumulative-hour propagation.

4.5. Rigorous feasibility theorem for C1–C10

We now state the corrected sufficiency result for the type-A maintenance model. Consider the constraint family (3)–(5), (8), (10)–(15), (20)–(24), and the stated variable domains.

Theorem 1 (Physical feasibility under corrected C1–C10). *Any solution satisfying the above constraints, with x , z , and y binary and h continuous, induces for each aircraft j a physically executable trajectory over the planning horizon: flights assigned to j are temporally non-overlapping, satisfy airport continuity and turn time, and admit a day-by-day maintenance schedule satisfying capacity and hour-spacing rules.*

Proof. Fix an aircraft j and let $S_j = \{i \in \mathcal{F} : x_{ij} = 1\}$ ordered by nondecreasing departure time.

(i) Flight assignment and continuity. By (3), every flight is assigned to exactly one aircraft. By (4)–(5), each departure assigned to j has sufficient prior balance at its departure airport, with the initial unit at a_j^0 . Therefore airport continuity holds along S_j .

(ii) Temporal executability. For any overlapping pair $\{i_1, i_2\} \in \mathcal{O}$, (8) enforces $x_{i_1j} + x_{i_2j} \leq 1$. Hence no two flights assigned to j overlap once turn time is included. Combined with (i), this yields a realizable single-aircraft flight timeline.

(iii) Maintenance event consistency. By (14), maintenance trigger variables can only be activated on assigned flights. By the aggregation equality (13), each day's maintenance trigger is represented by the single aggregate indicator y_{jd} ; by the daily cap (15), at most one maintenance event is attached to aircraft j on day d .

(iv) Maintenance capacity and blocking. By (15), airport-day capacities are respected. By (13), if maintenance is triggered after flight i on day d , then any subsequent same-day flight is forbidden for the same aircraft. Under the stated overnight-duration assumption, the maintenance action is operationally executable.

(v) Hour/day maintenance admissibility. By (20)–(24), the cumulative flight-time state increases by the flying time assigned each day, resets after a maintenance night, and never exceeds $T_{\max}^A$.

Steps (i)–(v) show that every aircraft has a consistent, non-overlapping flight sequence with feasible maintenance placement and resource usage. Therefore the global solution is physically feasible. $\square$

4.6. Issues of day-indexed maintenance constraints C1-C10

Usage of day-indexed maintenance variables y_{jd} and z_{ijd} is operationally intuitive, gives clear modeling semantics, but it has the following drawbacks:

- The day-indexed formulation requires a large number of big- M constants $M_{dd'}$ to cover all day pairs (d, d') with $2 \leq d' - d \leq d_{\max} - 1$. This

is a combinatorial number of rows, and the big- M values are necessarily loose because they must cover all possible flight-hour accumulations over the horizon.

- The day-indexed formulation does not propagate the flight-hour state exactly. The big- M relaxation allows the flight-hour counter to be reset prematurely, leading to potential violations of the intended maintenance intervals. For example, if a maintenance night is scheduled on day d and the next maintenance night is scheduled on day d' , the big- M constraints only enforce that the cumulative flight hours between d and d' do not exceed $T_{\max}^A$, but they do not prevent excessive flight hours before day d .
- The day-indexed formulation forces a specific maintenance schedule on each day for a given time zone, thus allowing to have two different maintenance checks within 24 hrs, but not in two different days within the same 24-hr period.
- The day-indexed formulation does not allow for a natural integration with the ordered flight formulation, which is more efficient and compact. The ordered flight formulation allows for a more direct representation of the maintenance events in relation to the flight sequence, leading to a more efficient model.
- The original cumulative-flight-time constraint (C11) in the day-indexed formulation is flawed, as it does not correctly enforce the flight-hour limit between two consecutive maintenance nights. This flaw can lead to infeasible solutions that violate the intended maintenance intervals.

4.7. Constraint (11): original formulation and its flaw

The original paper's cumulative-flight-time constraint (16) is intended to enforce that the cumulative flight hours between two consecutive maintenance nights do not exceed $T_{\max}^A$. However, as shown in Lemma 1, this constraint can fail to enforce the intended maintenance intervals in certain cases, particularly when there is no maintenance scheduled at the start of the interval.

Lemma 1 (Flaw in (16)). *Consider two consecutive maintenance events at days d and d' with no check between them ($y_{jr} = 0$ for $r \in (d, d')$). Then the RHS of (16) equals $T_{\max}^A + M_{dd'}(2 - 1 - 1) + M_{dd'} \cdot 0 = T_{\max}^A$, which correctly imposes the flight-hour limit between the two checks.*

However, if $y_{jd} = 0$ (no maintenance at the start day) and $y_{jd'} = 1$ (maintenance only at the end day), the RHS becomes $T_{\max}^A + M_{dd'}(2 - 0 - 1) = T_{\max}^A + M_{dd'}$, which is vacuously large and imposes no limit on cumulative flight hours before the first check of the period. Hence the constraint fails to enforce the check-interval bound for the opening sub-period.

Proof. Direct substitution of $y_{jd} = 0$, $y_{jd'} = 1$, $\sum_r y_{jr} = 0$ into (16) gives the stated RHS. Since $M_{dd'}$ is chosen as the maximum possible cumulative flight time over the horizon (a large positive constant), the resulting constraint is trivially satisfied regardless of the actual flight hours flown. $\square$

4.8. Constraint (11): endpoint-split strengthening and its limitation

An endpoint-split strengthening considered for replacing (16) consists of two constraints, each relaxed by one endpoint:

$$\sum_{i \in F_{d,d'}} x_{ij} \tilde{t}_i \leq T_{\max}^A + M_{dd'} y_{jd} + M_{dd'} \sum_{r \in [d+1, d'-1]} y_{jr}, \quad (17)$$

$$\sum_{i \in F_{d,d'}} x_{ij} \tilde{t}_i \leq T_{\max}^A + M_{dd'} y_{jd'} + M_{dd'} \sum_{r \in [d+1, d'-1]} y_{jr}, \quad (18)$$

for all $j \in \mathcal{P}$, $(d, d') \in \mathcal{D}^2$ with $2 \leq d' - d \leq d_{\max} - 1$, $d < n_d$.

Lemma 2 (The endpoint split is not sufficient). *With $F_{d,d'}$ defined as the flights departing in $(d, d']$, constraints (17)–(18) do not necessarily bound all flying time between two consecutive maintenance nights.*

Proof. Let $T_{\max}^A = 10$, $M_{dd'} = 20$, $d_{\max} \geq 4$, and consider days $1, \dots, 4$ with $y_{j1} = y_{j4} = 1$ and $y_{j2} = y_{j3} = 0$. Assign 10 units of flying time on day 2 and 10 units on day 4. For $(d, d') = (1, 3)$, the accumulated time is 10 and the second split inequality is active. For $(2, 4)$, the accumulated time is 10 and the first split inequality is active. For $(1, 4)$, the accumulated time is 20, but both inequalities have right-hand side $T_{\max}^A + M_{14} = 30$ because both endpoints contain checks. Thus every split inequality is satisfied although 20 units are flown between the consecutive checks, exceeding the limit 10. $\square$

The original and split formulations are therefore not ordered by strength: the original row is active when both endpoints contain checks, whereas the split rows are active when at least one corresponding endpoint indicator is zero. Neither pattern alone gives an exact cumulative counter.

Remark 1 (Implementation status). The default builder in `src/model.py` constructs the endpoint-split inequalities (17)–(18); the original single-row form (16) remains available as an explicit, opt-in `use_paper_c13` flag for controlled comparison, and an interval-specific tight big- M (`use_tight_c13_m`)

is available as a formulation-preserving strengthening of either row family. The computational study in Section 6 numerically confirms both Lemma 1 and Lemma 2 against the real built constraint objects of both forms, at two adversarial check patterns on the same four-day test instance: with checks at both window endpoints, all 42 split rows are satisfied while the true cumulative flying time exceeds $T_{\max}^A$ by a factor of two, and the single paper-form row correctly rejects it; with only the end-day checked, the paper-form row is satisfied at the same violation while all 42 split rows correctly reject it. The event-based implementation in `src/event_model.py` uses a flight-sequence state recursion analogous to the exact formulation below.

Four-day counterexample and implemented repair.. The failure and its repair can be traced completely on one aircraft over four days. Let $T_{\max}^A = 10$, $M_{dd'} = 20$, $v_{j1} = v_{j3} = 0$, and $v_{j2} = v_{j4} = 10$, with maintenance after the flights of days 1 and 4: $y_{j1} = y_{j4} = 1$ and $y_{j2} = y_{j3} = 0$. The check on night 1 resets the counter before day 2, whereas the check on night 4 occurs only after the day-4 flying and therefore cannot legalize those hours. Nevertheless, every endpoint-split row is satisfied. For $(d, d') = (1, 3)$, the interval $(1, 3]$ contains 10 units and the row anchored at day 3 has right-hand side 10. For $(2, 4)$, the interval $(2, 4]$ also contains 10 units and the row anchored at day 2 has right-hand side 10. For $(1, 4)$, the interval contains all 20 units, but each row has right-hand side $10 + 20 = 30$ because its corresponding endpoint contains a check. Thus the split model accepts 20 units between consecutive maintenance nights. Under the exact recursion (20)–(24), the same data give $h_{j2} = 10$, $h_{j3} = 10$, and $h_{j4} = 20$; the last value violates $h_{j4} \leq T_{\max}^A = 10$. In the code, the preserved class `LegacyEndpointSplitMILPScheduler` is identi-

ified as `legacy_endpoint_split`. The separate `EventMILPScheduler`, identified as `event_exact_state`, implements the equivalent event-level repair: its state is initialized with the pre-horizon hours, propagated by each selected successor flight, reset to the next flight's duration only when a qualifying maintenance event follows the preceding flight, and bounded by the applicable A/B threshold at every event. Unselected successor arcs are deactivated with the valid bound $T_{\max}^c + \tilde{t}_\ell$, while an unused source arc is deactivated with $\Phi_j^c + \tilde{t}_i$; using only $T_{\max}^c$ in either role can create false infeasibility. The comparison harness `experiments/compare_formulations.py` selects these two identifiers explicitly and rejects unknown identifiers, so the legacy and exact-state rows can no longer be produced by the same builder.

4.9. Constraint (11): exact cumulative-state formulation

Define the flying time assigned to aircraft j on day d by

$$v_{jd} = \sum_{i \in \mathcal{F}: d_i = d} \tilde{t}_i x_{ij}. \quad (19)$$

Let $h_{jd} \geq 0$ be the accumulated type-A flying time immediately after the flights of day d and before maintenance that night. Set $h_{j0} = \Phi_j^A$ and $y_{j0} = 0$. For consecutive days, impose

$$h_{jd} \geq h_{j,d-1} + v_{jd} - M^H y_{j,d-1}, \quad (20)$$

$$h_{jd} \leq h_{j,d-1} + v_{jd} + M^H y_{j,d-1}, \quad (21)$$

$$h_{jd} \geq v_{jd}, \quad (22)$$

$$h_{jd} \leq v_{jd} + M^H (1 - y_{j,d-1}), \quad (23)$$

$$0 \leq h_{jd} \leq T_{\max}^A, \quad \forall j \in \mathcal{P}, d \in \mathcal{D}, \quad (24)$$

where $M^H \geq T_{\max}^A$ is valid because $h_{j,d-1} \leq T_{\max}^A$. If $y_{j,d-1} = 0$, the first two inequalities give $h_{jd} = h_{j,d-1} + v_{jd}$. If $y_{j,d-1} = 1$, the reset inequalities give $h_{jd} = v_{jd}$. The upper bound then enforces the flight-hour threshold on every day, including the flights operated immediately before a night check.

4.10. Pre-horizon accumulated hours

The original paper does not account for flying time accumulated before the planning horizon. In the cumulative-state formulation this history is included exactly through the initial condition

$$h_{j0} = \Phi_j^A, \quad 0 \leq \Phi_j^A \leq T_{\max}^A, \quad \forall j \in \mathcal{P}. \quad (25)$$

The condition is required for every aircraft, including $\Phi_j^A = 0$; no separate opening-window big- M inequality is needed.

4.11. Integrality constraints

$$\begin{aligned} x_{ij} \in \{0, 1\}, \quad z_{ijd} \in \{0, 1\}, \quad y_{jd} \in \{0, 1\}, \quad (i, j) \in \mathcal{F} \times \mathcal{P}, \\ d \in \mathcal{D}, \quad i \in \bigcup_{m \in \mathcal{M}} F_d^\downarrow(m) \text{ for } z_{ijd}. \end{aligned} \quad (26)$$

The variables h_{jd} are continuous and satisfy $0 \leq h_{jd} \leq T_{\max}^A$ as imposed in (24).

As we see, the fixes to the original paper's maintenance model are twofold: (i) the C2–C4 correction, which ensures that the flight assignment is physically feasible, and (ii) the C11 correction. Also, the C11 correction is implemented in two ways: (i) the endpoint-split strengthening, which is a partial fix, and (ii) the exact cumulative-state formulation, which is a complete fix.

The exact cumulative-state formulation correctly propagates the flight-hour state and enforces the maintenance intervals as intended. The complexity of the corrected model is higher than the original model, because we need to track the cumulative flight hours explicitly that require more $O(n_f D)$ variables, but it is necessary to ensure that the solutions are physically feasible and respect the maintenance requirements.

5. Ordered Event-Based Reformulation for Type-A Maintenance

In this paper, we present an ordered event-based model for type-A maintenance. It removes the day index and does not introduce the standalone successor-arc event formulation. Flights are processed in nondecreasing departure-time order, and maintenance is represented by a binary event attached to the flight that triggers it. Thus the design-note variables x_{ij} and z_{ij} are represented below through an ordered position index r that maps to a flight index i_r .

5.1. Ordered event representation

For flight i , let o_i and a_i be its departure and arrival airports, p_i its flying time, and $t_i^\downarrow$ and $t_i^\uparrow$ its departure and arrival timestamps. For aircraft j , let a_j^0 be its initial airport.

Define the ordered-position index set

$$\mathcal{R} = \{1, \dots, n_f\},$$

and let $(i_1, \dots, i_{n_f})$ be flights sorted by $(t_i^\downarrow, i)$. Position $r \in \mathcal{R}$ refers to flight i_r . This is the only role of r : it is an order index over flights. Define

$$\mathcal{R}^M = \{r \in \mathcal{R} : i_r \in \mathcal{F}^M\},$$

and use the following variables:

- $x_{rj} \in \{0, 1\}$: aircraft j operates flight i_r ;
- $z_{rj} \in \{0, 1\}$: a type-A maintenance event is performed after flight i_r on aircraft j ;
- $h_{rj} \geq 0$: cumulative flight time of aircraft j immediately after processing position r .

Because $r \leftrightarrow i_r$ is one-to-one, the relation to the original notation is direct: $x_{rj} = x_{i_r j}$ and $z_{rj} = z_{i_r j}$.

A maintenance event after flight i_r is assignment-linked, starts at $t_{i_r}^\uparrow$, and occupies the aircraft for duration δ_A at a maintenance-capable arrival airport.

The corrected airport-continuity, turn-time, and overlap constraints remain active on the assignment variables. In particular, an assignment is allowed only when it can be connected to the aircraft's preceding assigned flight and when the required maintenance duration leaves enough time before the next assigned departure.

The objective is

$$\min \sum_{r \in \mathcal{R}} \sum_{j \in \mathcal{P}} c_{i_r j} x_{rj} + \sum_{r \in \mathcal{R}^M} \sum_{j \in \mathcal{P}} g_{i_r j} z_{rj}. \quad (27)$$

5.2. Prefix-state propagation

Let T_A be the maximum flying time between type-A events and let Φ_j be the pre-horizon accumulated flying time of aircraft j . At every ordered position, an unassigned flight carries the state forward, an assigned flight adds its duration, and an event resets the state after its triggering flight.

Using valid position-specific big- M values M_{rj} , the transition is represented by

$$h_{rj} \geq h_{r-1,j} - M_{rj}x_{rj}, \quad (28)$$

$$h_{rj} \leq h_{r-1,j} + M_{rj}x_{rj}, \quad (29)$$

$$h_{rj} \geq h_{r-1,j} + p_{i_r} - M_{rj}(1 - x_{rj}) - M_{rj}z_{rj}, \quad (30)$$

$$h_{rj} \leq h_{r-1,j} + p_{i_r} + M_{rj}(1 - x_{rj}) + M_{rj}z_{rj}, \quad (31)$$

$$h_{rj} \geq p_{i_r}z_{rj}, \quad (32)$$

$$h_{rj} \leq p_{i_r}z_{rj} + T_A(x_{rj} - z_{rj}) + M_{rj}(1 - x_{rj}), \quad (33)$$

$$h_{r-1,j} + p_{i_r} \leq T_A + M_{rj}(1 - x_{rj}), \quad (34)$$

The first two rows carry the state at an unassigned position. The next two rows impose accumulation for an assigned flight unless an event is selected. The fifth and sixth rows together give the exact post-event state: $h_{rj} = p_{i_r}$ when $z_{rj} = 1$. The final row prevents a maintenance event from repairing an arrival that already exceeds T_A . The initial state is $h_{0j} = \Phi_j$.

Events satisfy

$$z_{rj} \leq x_{rj}, \quad \forall r \in \mathcal{R}^M, \forall j \in \mathcal{P}, \quad (35)$$

and timestamp-based incompatibility rows prevent an event from being followed by a departure that occurs before the maintenance duration and turn time have elapsed. These rows preserve physical route feasibility while avoiding a separate successor-variable family. Capacity is enforced directly by event-overlap inequalities at each airport and timestamp breakpoint. This is equivalent to maintaining an airport occupancy state $u_{a,\theta}$ and imposing $u_{a,\theta} \leq \text{mcap}_{a,\theta}$, but it requires no additional occupancy variable.

The ordered event rows correspond to the day-indexed maintenance constraints: event assignment is C9, duration blocking is C8, and timestamp-overlap capacity is C10. The at-most-one-event-per-aircraft-per-day rule is retained explicitly as

$$\sum_{r \in \mathcal{R}: \lfloor t_{i_r}^\uparrow / 1440 \rfloor = d} z_{rj} \leq 1, \quad \forall j \in \mathcal{P}, d \in \mathcal{D}, \quad (36)$$

which is the event-indexed counterpart of C14.

Table 3 records the correspondence with the numbered day-indexed requirements. Some legacy rows disappear because their variables are eliminated; this is a formulation change, not an omitted feasibility condition.

Proposition 3 (Ordered prefix-state validity). *Assume the corrected route constraints define a physically feasible integral assignment, every event is attached to an assigned flight at a maintenance-capable airport, and the supplied initial state is valid. Then the ordered prefix-state formulation satisfies the type-A flight-hour limit.*

Proof. Induct on the ordered flight positions. An unassigned position carries the state unchanged. An assigned position adds its flying time unless an event is selected, in which case the two reset rows set the state to the current flight’s duration. The final transition row and the upper state bound prevent the threshold from being exceeded before a reset. The induction starts from Φ_j , so every assigned route satisfies the type-A limit. $\square$

5.3. Why the ordered formulation is the final method

The ordered formulation combines event-triggered maintenance with a global flight order and aircraft-local prefix states. It removes the day-pair

Table 3: Coverage of the numbered C1–C14 requirements in the ordered A-only formulation.

Requirement	Day-indexed role	Ordered A-only counterpart
C1	Flight coverage	Coverage equation on x_{rj} (equivalently $x_{i_{rj}}$).
C2–C3	Airport continuity and turn time	Corrected equipment-flow balance on assignment variables.
C4	Physical non-overlap	Explicit overlap inequalities on x .
C5–C7	Domains and maintenance eligibility	Binary domains; z_{rj} is created only for eligible trigger flights and satisfies $z_{rj} \leq x_{rj}$.
C8	Maintenance blocks later flights	Timestamp blocking using $\delta_A + \Delta t$.
C9	Maintenance requires assignment	The event-assignment inequality above.
C10	Maintenance capacity	Direct timestamp-overlap capacity rows, equivalent to an airport occupancy state.
C11	Link event triggers to a daily indicator	Daily indicator is eliminated; it is recoverable from the events on each day.
C12	Calendar-time spacing	Inactive for type-A-only maintenance; C13 enforces the applicable hour limit.
C13	Cumulative flight hours	Exact prefix-state transitions, reset bounds, and pre-check bound.
C14	At most one event per aircraft per day	Explicit daily event inequality (36).

accumulation ambiguity of Constraint (11), avoids successor-variable expansion, and keeps the physical route constraints needed for valid tail assignments. For type-A maintenance, C12’s calendar-spacing role is inactive: the flight-hour limit is enforced by the exact prefix state in C11. The computational study in Section 6 evaluates this formulation against the legacy A-only model on matched horizon and fleet-size grids.

Table 4: Numeric verification of the C11 flaw against the production constraints and exact-state replay.

Scenario	Paper C13	Endpoint split	Exact state
Both endpoints checked	rejects	accepts (wrong)	rejects
Only end checked	accepts (wrong)	rejects	rejects

6. Computational Study

6.1. Implementation and environment

All models are implemented in Python using Pyomo. The reported exact solves use unrestricted IBM ILOG CPLEX 12.6 for AMPL through Pyomo’s `cplexamp` interface. The deterministic instance generator and all benchmark scripts are included in the companion repository. Every reported solution is checked independently by replaying the aircraft route and its maintenance counter.

6.2. Constraint (11) verification

The original cumulative-flight-time row and the natural endpoint-split replacement were evaluated on the hand-built C11 counterexample. The actual production constraint objects admit complementary violating patterns: the original row is relaxed when only one endpoint is checked, while the split replacement is relaxed when both endpoints carry checks. The exact cumulative state formulation rejects both patterns. This directly confirms the algebraic analysis in Sections 4.7 and 4.8.

Table 5: A-only solve-time comparison by planning horizon. Positive gain means ordered A-event is faster than legacy A-only.

h	Wall L	Wall O	Wall gain (%)	CPU L	CPU O	CPU gain (%)	Vars L/O	Cons
7	0.809661	0.797610	+1.49	0.437317	0.436940	+0.09	1910/1250	3498
15	1.075419	0.818485	+23.89	0.537919	0.437708	+18.63	3370/1500	7494
21	1.055696	0.827527	+21.61	0.538825	0.438074	+18.70	3370/1500	7494
30	1.074072	0.820306	+23.63	0.537266	0.436448	+18.77	3370/1500	7494
Mean	–	–	+18.70	–	–	+14.73	–	–

6.3. Ordered type-A event formulation

The ordered implementation removes explicit successor variables and propagates an aircraft-local prefix state through flights sorted by departure time. A maintenance event is attached to its triggering flight, and the pre-check row prevents an event from repairing an already overdue arrival. This is the latest A-only method evaluated below. The ordered formulation agrees with the legacy A-only model on every feasible instance reported here.

6.3.1. A-only timing versus planning horizon

We compared legacy A-only and ordered A-event models on the same feasible instances with $|P| = 10$ and $h \in \{7, 15, 21, 30\}$ using unrestricted CPLEX. The complete measurements are in `results/tables/formulation_a_comparison_latest`.

The ordered A-event method is faster on all four horizons. Its mean gain is +18.70% in wall time and +14.73% in solver CPU time; for $h \geq 15$, the CPU gain is approximately 19% on this family.

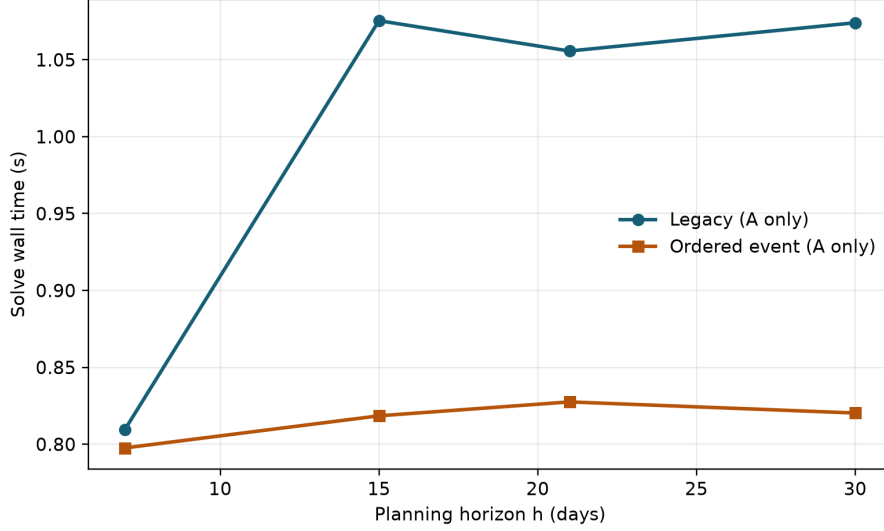


Figure 1: A-only solve wall time versus planning horizon.

Table 6: A-only CPU-time scaling with fleet size at $h = 7$. Positive gain means ordered A-event is faster than legacy A-only.

$ P $	CPU Legacy	CPU Ordered	Gain (%)	Vars L/O	Cons. L/O	Obj. Legacy	Obj. Ordered
5	0.335890	0.335151	+0.22	320/125	748/465	4 839	4 839
10	0.336071	0.336778	-0.21	990/500	2 128/1 873	9 627	9 627
15	0.336498	0.333956	+0.76	1 950/1 125	3 938/4 283	14 341	14 341
20	0.437365	0.335564	+23.28	3 300/2 000	6 248/7 511	19 092	19 092
Mean gain	–	–	–	+6.01	–	–	–

6.3.2. A-only CPU scaling versus fleet size

We fixed $h = 7$ and compared the legacy A-only and ordered A-event formulations on feasible instances with $|P| \in \{5, 10, 15, 20\}$. All eight runs were optimal and objective-matched. Results are in `results/tables/formulation_a_fleet_scaling_1`.

The ordered formulation has a mean CPU-time gain of +6.01% over legacy on this four-point grid and uses fewer variables at every fleet size. The gain is not monotone: it is slightly negative at $|P| = 10$ and is largest

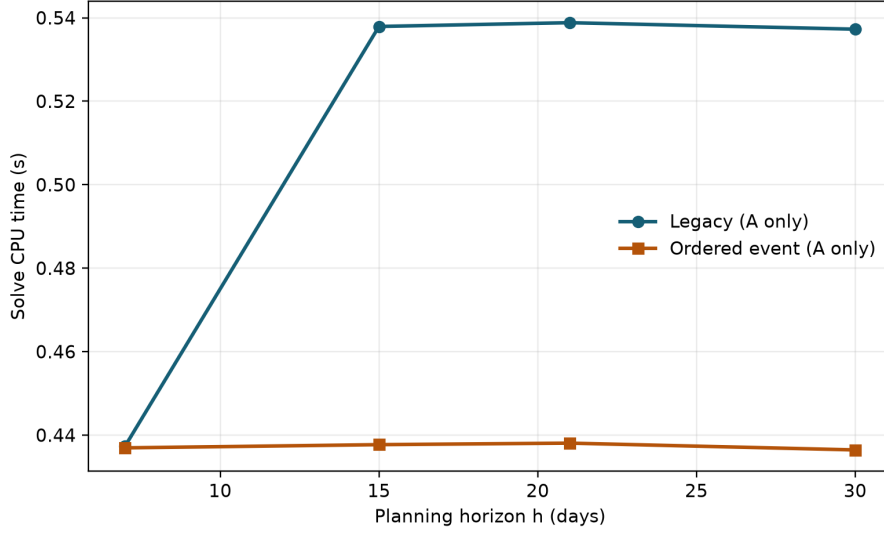


Figure 2: A-only solve CPU time versus planning horizon.

at $|P| = 20$. These are single-run measurements and should be interpreted as descriptive rather than as a universal scaling law.

6.4. Summary and limitations

The corrected type-A model rejects the C11 counterexample, while the ordered A-event implementation preserves objective parity and reduces variable counts on the tested feasible grids. Its timing advantage is strong in the A-only horizon experiment and is positive on average in the fleet-size experiment, but the benchmark contains one instance per cell. The results therefore support the ordered formulation as a compact and promising A-only method, not as a solver-independent or distribution-independent runtime guarantee.

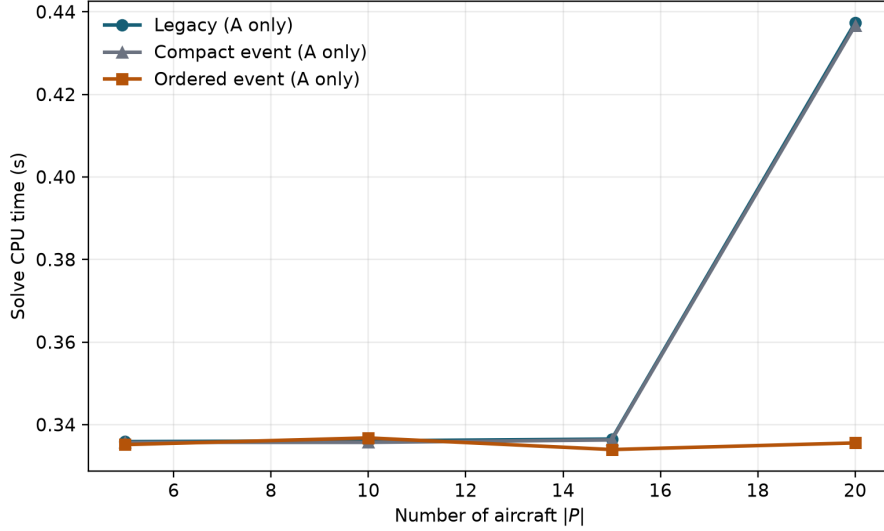


Figure 3: A-only solve CPU time versus fleet size at fixed horizon $h = 7$.

7. Conclusions and Future Work

This paper revisited the compact Tail Assignment Problem formulation of Khaled et al. (2018) and made three focused contributions. First, we showed that the basic continuity and turn-time inequalities admit a simultaneous-departure corner case and closed it with explicit non-overlap constraints. Second, we proved that the paper’s cumulative-flight-time Constraint (11) can relax a maintenance interval incorrectly and that the natural endpoint split is also insufficient. We provided an exact cumulative-state correction and verified the failure against the production constraint objects. Third, we developed a day-free event-based reformulation for type-A maintenance and an ordered prefix-state implementation.

The ordered event formulation attaches maintenance to its triggering flight and propagates the flight-hour counter along the aircraft’s selected

route. The ordered implementation removes explicit successor variables while preserving route feasibility and the pre-check condition. On the feasible A-only instances studied here, the ordered method matched the legacy optimum in every run, used fewer variables at every fleet size, and gave a mean wall-time gain of 18.70% and mean solver CPU-time gain of 14.73% across the tested horizon family. At fixed $h = 7$, the fleet-size experiment gave a mean CPU-time gain of 6.01% over legacy across $|P| \in \{5, 10, 15, 20\}$, although the gain was non-monotone and concentrated at the largest fleet.

These computational results are deliberately scoped. They use deterministic synthetic feasible instances, one instance per parameter cell, and a single unrestricted CPLEX configuration. The results establish objective parity and indicate useful compactness and timing behavior for the tested A-only family; they do not establish a universal speedup across solvers, instance distributions, or larger benchmark sets.

Reproducibility. The project contains the model implementations, deterministic instance generator, benchmark scripts, result tables, and plotting scripts needed to reproduce the reported snapshots. The legacy, event, and ordered A-only builders remain separately selectable so future comparisons can distinguish model semantics from implementation choices.

Future directions

- **Larger A-only benchmark families:** repeat the horizon and fleet-size experiments over multiple seeds and heterogeneous timetable structures, reporting solver variability and confidence intervals.

- **Integrated fleet assignment and routing:** extend the corrected compact model while retaining physically executable route constraints.
- **Robust and recoverable variants:** study uncertainty in flight times, disruptions, and maintenance availability under the exact state formulation.
- **Heuristic methods:** develop constructive and local-search methods for the corrected A-only model and compare them with the exact formulations on larger instances.

References

- Barnhart, C., Belobaba, P., Odoni, A.R., 2003. Applications of operations research in the air transport industry. *Transportation Science* 37, 368–391.
- Barnhart, C., Boland, N.L., Clarke, L.W., Johnson, E.L., Nemhauser, G.L., Shenoi, R.G., 1998. Flight string models for aircraft fleetting and routing. *Transportation Science* 32, 208–220. doi:10.1287/trsc.32.3.208.
- Borndörfer, R., Dovica, I., Nowak, I., Schickinger, T., 2010. Robust tail assignment. Technical Report. Konrad-Zuse-Zentrum für Informationstechnik Berlin.
- Clarke, L., Johnson, E., Nemhauser, G., Zhu, Z., 1997. The aircraft rotation problem. *Annals of Operations Research* 69, 33–46. doi:10.1023/A:1018945415148.

- Clarke, L.W., Hane, C.A., Johnson, E.L., Nemhauser, G.L., 1996. Maintenance and crew considerations in fleet assignment. *Transportation Science* 30, 249–260.
- Cordeau, J.F., Stojković, G., Soumis, F., Desrosiers, J., 2001. Benders decomposition for simultaneous aircraft routing and crew scheduling. *Transportation Science* 35, 375–388.
- Desaulniers, G., Desrosiers, J., Dumas, Y., Solomon, M.M., Soumis, F., 1997. Daily aircraft routing and scheduling. *Management Science* 43, 841–855.
- Froyland, G., Maher, S.J., Wu, C.L., 2013. The recoverable robust tail assignment problem. *Transportation Science* 48, 351–372.
- Gopalan, R., Talluri, K.T., 1998. Mathematical models in airline schedule planning: A survey. *Annals of Operations Research* 76, 155–185.
- Grönkvist, M., 2005. The tail assignment problem. Ph.D. thesis. Chalmers, Göteborg University.
- Hane, C.A., Barnhart, C., Johnson, E.L., Marsten, R.E., Nemhauser, G.L., Sigismondi, G., 1995. The fleet assignment problem: Solving a large-scale integer program. *Mathematical Programming* 70, 211–232.
- Haouari, M., Shao, S., Sherali, H.D., 2012. A lifted compact formulation for the daily aircraft maintenance routing problem. *Transportation Science* 47, 508–525.
- Khaled, O., Minoux, M., Mousseau, V., Michel, S., Ceugniet, X., 2018. A compact optimization model for the tail assignment problem. *Euro-*

- pean Journal of Operational Research 264, 548–557. doi:10.1016/j.ejor.2017.06.045.
- Klabjan, D., 2005. Large-scale models in the airline industry, in: Desaulniers, G., Desrosiers, J., Solomon, M.M. (Eds.), Column Generation. Springer US, pp. 163–195.
- Lacasse-Guay, E., Desaulniers, G., Soumis, F., 2010. Aircraft routing under different business processes. Journal of Air Transport Management 16, 258–263.
- Lavoie, S., Minoux, M., Odier, E., 1988. A new approach for crew pairing problems by column generation with an application to air transportation. European Journal of Operational Research 35, 45–58.
- Levin, A., 1971. Scheduling and fleet routing models for transportation systems. Transportation Science 5, 232–255.
- Liang, Z., Chaovalitwongse, W.A., 2012. A network-based model for the integrated weekly aircraft maintenance routing and fleet assignment problem. Transportation Science 47, 493–507.
- Liang, Z., Chaovalitwongse, W.A., Huang, H.C., Johnson, E.L., 2011. On a new rotation tour network model for aircraft maintenance routing problem. Transportation Science 45, 109–120.
- Mercier, A., Cordeau, J.F., Soumis, F., 2005. A computational study of Benders decomposition for the integrated aircraft routing and crew scheduling problem. Computers & Operations Research 32, 1451–1476. doi:10.1016/j.cor.2003.11.013.

- Minoux, M., 1984. Column generation techniques in combinatorial optimization, a new application to crew pairing problems, in: Proceedings of the AGIFORS Symposium, Strasbourg, France.
- Sarac, A., Batta, R., Rump, C.M., 2006. A branch-and-price approach for operational aircraft maintenance routing. *European Journal of Operational Research* 175, 1850–1869.
- Sherali, H.D., Bish, E.K., Zhu, X., 2006. Airline fleet assignment concepts, models, and algorithms. *European Journal of Operational Research* 172, 1–30.